Tunwall: A Userspace Stateful Firewall

Giselle Wu and Eva Tate
CS 60 Final Project: Requirements Specification
https://github.com/oooogiselle/Networks-final-project

1. Overview

Almost every home router and company network has a stateful firewall at its edge. People inside the network can open connections to the outside, replies to those connections are let back in, and everything else is dropped. On Linux, this is handled inside the kernel (netfilter/conntrack), so it is hard to see what is actually going on. For our project we will build our own stateful firewall in userspace, in C. Our program sits between an inside (trusted) network and an outside (untrusted) network using two Linux TUN devices. The kernel gives our program every packet that crosses between the two networks as raw IPv4 bytes. For each packet we parse and validate the headers, look up or create connection state, check a list of rules, and then either forward the packet to the other side or drop it.

When we are done, a client inside the network should be able to load web pages, use DNS, and ping outside hosts. Outside hosts should not be able to start connections to inside hosts unless a rule allows it, and an nmap scan from outside should show ports as filtered. We will also compare our throughput and latency against normal Linux kernel forwarding and explain where the extra overhead comes from.

2. Who It Serves

The main users would be people running a small network, like a home lab, a dorm, or a small office, who want a default-deny firewall with a simple rule file and logs that say why each packet was allowed or dropped. It is also useful as a teaching tool. Because every decision is logged with a reason, someone can watch how header parsing, checksums, and the TCP connection lifecycle turn into a security policy, which is normally hidden inside the kernel.

3. Major Entities

Entity	Description
Packet	A raw IPv4 packet read from a TUN device. The parser pulls out the header fields without copying the packet.
Interfaces	tun-in faces the inside network and tun-out faces the outside network. The interface a packet arrives on tells us its direction.
Flow	One connection, identified by its 5-tuple (protocol, source IP, source port, destination IP, destination port). For ICMP echo we use the ICMP identifier instead of ports. Each flow stores its state, which side started it, packet and byte counts, and when it expires.
Flow table	A hash table of flows. Keys are put in a standard order so both directions of a connection find the same entry. It has a fixed maximum size (65,536 flows by default).

Rules	An ordered list of allow/deny rules that match on direction, protocol, source and destination address (CIDR), and destination port. The first matching rule wins, and the default is deny.
Hosts	Inside clients on 10.0.1.0/24 and outside servers and attackers on 10.0.2.0/24, each group in its own Linux network namespace.

4. Requirements

Functional requirements

1. **Parsing and validation.** Parse IPv4, TCP, UDP, and ICMP headers and check lengths and checksums (including the TCP/UDP pseudo-header). Malformed packets are dropped before they can create or change any state. For this version, IP fragments and IPv6 packets are dropped.
2. **Rule engine.** Rules are loaded from a text file when the program starts. If the file has a syntax error, the program prints the line number and refuses to start. Example rule file:

```
# action  dir  proto  src           dst        ports
allow     out  tcp    10.0.1.0/24   any        80,443
allow     out  udp    10.0.1.0/24   any        53
allow     out  icmp   10.0.1.0/24   any        echo
allow     in   tcp    any           10.0.1.3   22
# anything not matched is denied
```

3. **Stateful filtering.** The first packet of a new connection is checked against the rules (slow path). If it is allowed and is a valid first packet (a TCP SYN, a UDP datagram, or an ICMP echo request), we add a flow to the table. Later packets in either direction only need a hash table lookup and a state check (fast path). Reply traffic is only allowed if it matches an existing flow and arrives on the opposite interface.
4. **TCP state tracking.** We track SYN_SENT, SYN_RECV, ESTABLISHED, FIN_WAIT, TIME_WAIT, and CLOSED, and handle RST. Packets that do not make sense for the current state are dropped by default. Examples are a non-SYN packet with no flow (like an ACK scan), a SYN-ACK without an earlier SYN, and invalid flag combinations like SYN+FIN or the NULL and Xmas scans.
5. **Timeouts.** Each state has an idle timeout. For example, half-open TCP connections time out after 30 s, established connections after 1 hour, UDP flows after 30 to 120 s, and ICMP echo after 30 s. Expired flows are removed by a periodic sweep.
6. **ICMP.** Echo request and reply are tracked like a flow. ICMP error messages (destination unreachable, time exceeded) are only allowed if the packet header inside them matches an existing flow, so traceroute and path MTU discovery still work.
7. **Anti-spoofing.** A packet arriving from the outside that claims an inside source address is dropped, and so is a packet from the inside with a source address that is not in the inside network.
8. **Logging and stats.** We log when each flow opens and closes (5-tuple, duration, packets, bytes) and log drops with a reason (malformed, no matching rule, denied by rule, invalid state, spoofed, table full). Counters are printed when the program exits or receives SIGUSR1.
9. **Default deny.** Anything we do not understand or cannot handle is dropped. If the flow table is full, new connections are refused but existing ones keep working. If our program is not running, no traffic gets through at all.

Non-functional requirements

- Written in C for Linux, using only the standard C library, with no iptables or nftables rules doing the filtering for us.
- No memory allocation per packet. Our goal is at least 200 Mbit/s of TCP throughput and under 0.5 ms of added latency on the course VM, and throughput should not depend on the number of rules once a connection is established.
- The program should never crash on bad input. Truncated, oversized, and random packets are part of our tests.

5. Network Communication Plan

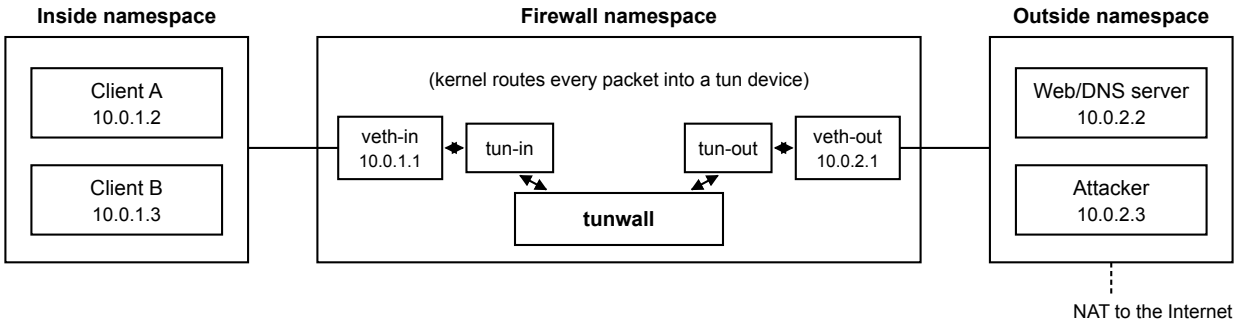


Figure 1: Test setup with three network namespaces on one Linux VM.

We will write a setup script that creates three network namespaces (inside, firewall, and outside) connected with veth pairs. In the firewall namespace we use policy routing (ip rule) so that every packet arriving from the inside is routed into tun-in, and every packet arriving from the outside is routed into tun-out. The kernel never forwards directly between the two sides, so our program is the only path. When our program writes an allowed packet to the opposite TUN device, the kernel routes it out the correct veth. The outside namespace also gets a NAT connection to the host so inside clients can reach real websites.

The program is a single-threaded loop that uses poll() on the two TUN file descriptors plus a timer for expiring flows. For each packet it does one read(), decides whether to allow it, and then either does one write() to the other TUN device or drops it.

Example: allowed packet. Client A runs curl to the web server. The SYN arrives on tun-in. There is no flow yet, so we check the rules, and the rule allowing outbound TCP on port 80 matches. We create a flow in SYN_SENT and write the packet to tun-out. The server's SYN-ACK arrives on tun-out, matches the flow in the reverse direction, and is forwarded to tun-in (the flow moves to SYN_RECV). The client's ACK makes the flow ESTABLISHED, and every later packet is handled by the fast path until the FINs close it.

Example: denied packet. The attacker sends a SYN to Client B on port 3389. It arrives on tun-out. There is no flow and no rule allowing inbound traffic to that port, so the default deny drops it and logs the reason. We do not send a RST, so the port looks filtered to the attacker. An ACK scan also gets dropped because it is not a valid first packet. Only port 22 on Client B, which our rules allow, shows up as open.

6. Testing and Evaluation

- **Unit tests** for the parser (valid and malformed packets), the TCP state machine (every state and event), the flow table (lookups in both directions, expiry, a full table), and the rule parser and matcher (first match wins, CIDR and port edge cases, default deny).
- **Integration tests** using the namespace script: curl and DNS work from the inside, inbound ping and inbound connections fail, nmap shows only the allowed port, idle flows expire, and killing the firewall stops all traffic.
- **GitHub Actions** runs the unit tests (with AddressSanitizer) and the integration tests on every push.
- **Performance:** iperf3 and ping comparing (a) normal kernel forwarding, (b) our firewall with one allow-all rule, and (c) our firewall with many rules. We will profile with perf to explain the overhead, which we expect to come from two system calls and two copies per packet, context switches, and the kernel being able to batch packets (GSO/GRO) while we handle one packet at a time.

For the demo, we will show that an inside client can download a real web page (checking that the SHA-256 hash matches), that unsolicited inbound connections are dropped, that an nmap scan only finds the allowed port, and our performance numbers compared to Linux.

7. Optional Advanced Features

If we have time, we will first add source NAT with port translation, so inside hosts share the firewall's outside address. This needs incremental checksum updates (RFC 1624). After that, we would add simple application identification that recognizes HTTP, TLS, and DNS by looking at the payload instead of trusting port numbers. Smaller extras include a reject action that sends a RST or ICMP unreachable, and reloading the rule file without restarting.

8. Division of Work and Timeline

Giselle: TUN devices and the namespace test setup, packet parser and checksums, the main I/O loop, and benchmarking/profiling.
Eva: flow table and timeouts, TCP state machine, rule file parsing and matching, and logging.
Both: the interface between the parser and the decision engine (which we will agree on first so we can work in parallel), integration tests, optional features, and the video.

When	Goal
Week 8	Repo setup, Makefile, and CI; namespace script; a firewall that forwards everything between the two TUN devices; parser and its unit tests.
Week 9	Flow table, rules, and TCP state machine working end to end; timeouts, ICMP handling, and logging; integration tests and benchmarks.
Week 10	Optional features (NAT first), final testing, and recording the video.

The biggest risk is getting the routing between the namespaces and TUN devices right, so we will do that first. If reaching the real Internet from the namespaces causes problems, we can use a local web server in the outside namespace instead.